

Code-Layout, Readability and Reusability

Date	01 October 2026
Team ID	SWTID-2026-3838
Project Name	LegalEase: AI-Powered Legal Document Generator
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/ tabs used throughout the code	Yes	Consistent indentation and spacing are followed throughout the Python, HTML, CSS and JavaScript code, making the code easy to read and maintain.
2	Proper File Structure	Files and folders are logically organized	Yes	The project is logically divided into ai_core, frontend, exporters, and API files such as main.py and routes.py.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Meaningful names such as doc_type, parties, terms, effective_date, edited_doc, original_doc, and preview_page clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as generate_document(), request_document(), save_changes(), parse_document(), render_paginated_html(), to_docx() and to_pdf() are descriptive and easy to understand.
5	Code Comments	Inline and block comments explain logic	Yes	The project contains module docstrings, function docstrings and inline comments explaining important operations such as API communication, page navigation, state management and document exporting.

6	Modular Design	Code is split into reusable functions/modules	Yes	The project is divided into separate modules for AI generation, API handling, frontend UI, styling, editing and document exporting.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Major unnecessary duplication is avoided, but some rendering/style logic is repeated in document preview functions and can be further refactored into common helper functions.
8	Error Handling	Exceptions and errors are handled carefully	Yes	Major unnecessary duplication is avoided, but some rendering/style logic is repeated in document preview functions and can be further refactored into common helper functions.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	generate_document()	Python + Gemini	AI document generation and fallback generation	High
2	document_utils.py	Python	Preview and document export	High
3	parse_document()	Python	HTML, DOCX and PDF processing	High
4	legalease_editor	Python	Document editing interface	High
5	load_css()	Python/CSS	Common Streamlit UI styling	High

Overall Code Quality Assessment:

Aspect	Rating	Comments
Code Layout & Structure	5	Files are properly separated into frontend, backend/API, AI core and exporters.
Readability	5	Meaningful variable/function names and consistent formatting are used.
Reusability	4	Functions such as document parsing and export are separated into reusable modules.
Documentation	4	Important functions and modules are explained, but documentation can still be expanded in some areas.
Overall Score	4	Overall, the project is well structured, readable, modular and maintainable.